Lab 12 – (Week of November 11th)

Individual Activity #1: Using the `PairOfDice.java` Instance Class Again

Last week's individual activity called upon you to make use of the given instance class, `PairOfDice` (that itself is a client of the `Die` class). Since not everyone seemed to fully grasp the point of that activity and since you have been provided with a solution, this week you are given another opportunity to complete a similar application program.

```
[3][3] and [3][1]
[5][6] and [3][6]
[3][5] and [4][6]
[6][3] and [6][1]
[5][3] and [2][6] both total to 8
```

The `PairOfDice.java` and `Die.java` classes are available for you to see.

Specifically, you are to provide all of the missing statements to complete the following Java application program that rolls two pair of dice until they come up with the same face value total (ass illustrated above). Provide your answer on this sheet in the space below, and hand it to your Lab Instructor when finished.

```
// This program simulates the rolling of two pair of dice until
// they come up with the same face value total.

public class SameTotal {
   public static void main(String[] args) {

}
```

Activity #2: The `TimeSpan.java` Instance Class

This is a continuation of one of last week's activities, and as such you are to pick up where you left off last week. In the event you are working with a different partner this week you are to pick up with the earliest common incomplete sub-activity.

Consider the given sample source programs, `TimeSpan.java`, `TimeSpanOperations.java`, and `MeanHHMM.java`, which together provide the ability to read a sequence of "time values" and determine and print their mean. As given, the "time values" are each of the form *hh:mm* where *hh* represents full hours and *mm* represents full residual minutes. For example, "`2:18`", "`2:47`" and "`2:09`".

The behavior of these program components can be seen in the sample execution given to the right. In this case the small data file, `Data0.txt`, which contains just the three time values given above, is used.

```
c:\C134L\Lab11>java MeanHHMM Data0.txt
1) 2:18 aka 2h18m accumulates to 2h18m
2) 2:47 aka 2h47m accumulates to 5h5m
3) 2:9 aka 2h9m accumulates to 7h14m
The mean time is 2h24m
```

The given `TimeSpan.java` source program is basically a straightforward extension of the example (of the same name) given in Chapter 8 of our course text. The `TimeSpanOperations.java` class is not an instance class, but instead presents a collection of static methods that are useful in working with values of this kind.

This activity consists of a series of "sub-activities" each of which contribute toward the eventual goal of refactoring the given components to represent and operate with time values that each express full hours, full residual minutes, and full residual seconds. That is, of the form, *hh:mm:ss*. Thus, the idea is to have finer precision and greater accuracy in "time span values".

```
2:18:51
2:47:42
2:09:29
```

Consider the data file, `Data0R.txt`, and the accompanying sample execution of the given client, `MeanHHMMSS.java` that makes use of the revised components. This illustrates the eventual goal of this entire activity.

```
c:\C134L\Lab11>java MeanHHMMSS Data0R.txt
1) 02:18:51 accumulates to 02:18:51
2) 02:47:42 accumulates to 05:06:33
3) 02:09:29 accumulates to 07:16:02
The mean time is 02:25:20
```

For the following sub-activities you are to retain the names of all of the files and the class components. Although these files will be modified, there is no need for you change the names of the classes and the files for this activity; the originals will remain on the CWS. You are to undertake the following sub-activities in the following order, getting as far as you can.

a) The `toString` method given in `TimeSpan.java` is awkward in how it expresses the time value using '`h`' and '`m`', and likewise inconsistent with the "parse" methods. So for this sub-activity you are to refactor `toString` to instead express the time value using colons. Thus, instead of "`2h18m`" it should return "`2:18`", and likewise instead of "`5h5m`" it should return "`5:05`". Note that minutes should always be expressed using two digits.

b) For the sake of greater completeness it would be appropriate to add `getHours()` and `getMinutes()` methods as "accessors/observers" to the given `TimeSpan` class. These can be readily tested by making a simple revision to `MeanHHMM.java` on line 48 as indicated by the comment, prefixed with "`//!!`".

It needs to be pointed out that from this point forward that as you work on the revisions and enhancements you may well find yourself in unstable situations. That is, you will have made valid revisions to one or more of the source code files, but may not have yet made the necessary corresponding revisions to other files and so not everything will compile. And, if things don't compile then you can't test. So, you need to thoughtfully and carefully plan each of the changes you make so that you maintain intellectual control over what you are trying to accomplish at that moment.

c) Another awkwardness is seen in the `MeanHHMM` program where the client calls upon the corresponding methods of the `TimeSpanOperations` class to "parse out", that is recognize and extract, each of the three parts (hours, minutes and seconds) from the string read from the file.

This awkwardness is easily resolved by adding an additional constructor in `TimeSpan` that takes a single string as its actual argument value and uses the time value expressed in that string as the value for the `TimeSpan` object. Thus, instead of the client parsing out the hours and minutes it is this new constructor that performs this task. In this way, the client does not need to have any awareness of the `TimeSpanOperations` class and the overall abstraction is stronger.

So, for this sub-activity you are to develop the additional constructor in `TimeSpan` and correspondingly revise and test `MeanHHMM.java`.

d) Since the essential enhancement of this activity is to extend the `TimeSpan` instance class to represent seconds in addition to hours and minutes, we will now move forward on this. So, at this point you are to transition away from `MeanHHMM.java` to using the client `MeanHHMMSS.java` instead to exercise your revisions. Note that there is no reason for you and you should not modify, `MeanHHMMSS.java`.

i. Since an additional parse method, `parseOutSecond`, is needed and both it and the other parse methods in `TimeSpanOperations` will need revision and testing, the program `parseDriver.java` is provided to you as a means of testing these revisions. This program is intended to be a throw-away program existing solely for the purpose of testing the parse methods. So now focus your attention on developing, revising and testing these three methods.

```
c:\C134L\Lab11>java parseDriver
Enter:>12:34:56
"12:34:56" yields 12 hours, and 34 minutes, and 56 seconds.
Enter:>   2   :  34  :   56
"12   :   34  :   56" yields 12 hours, and 34 minutes, and 56 seconds.
Enter:>1:2:3
"1:2:3" yields 1 hours, and 2 minutes, and 3 seconds.
Enter:>1234:56:7
"1234:56:7" yields 1234 hours, and 56 minutes, and 7 seconds.
Enter:>123456:78:90
"123456:78:90" yields 123456 hours, and 78 minutes, and 90 seconds.
Enter:>12:hello:bye
"12:hello:bye" yields 12 hours, and Exception in thread "main"
java.lang.NumberFormatException: For input string: "hello"
        at java.lang.NumberFormatException.forInputString(Unknown Source)
        at java.lang.Integer.parseInt(Unknown Source)
        at java.lang.Integer.parseInt(Unknown Source)
        at TimeSpanOperations.parseOutMinute(TimeSpanOperations.java:46)
        at parseDriver.main(parseDriver.java:29)
```

ii. Note that `TimeSpan` as given has a single instance variable, named `totalMinutes`, that obviously must be renamed `totalSeconds`. Of course, you'll have to modify the parameter lists and bodies of each of the effected methods, including those methods which you have revised in preceding sub-activities.

iii. The `TimeSpanOperations` class will need additional methods to deal with seconds (i.e. `getSecond()`), and it will also be necessary to modify the parameter lists (at least the formal names of the parameters) and the bodies of some of the existing methods accordingly.

iv. You'll note that `MeanHHMMSS.java` is shorter, more streamlined and more apparent than `MeanHHMM.java`. Part of the reason for this is because of the additional methods developed in the earlier sub-activities. For this sub-activity, you will need to declare and implement each of these additional methods in your revised `TimeSpan.java` class. They include:

- An additional mutator, `add`, that takes a `TimeSpan` object as its actual argument value, instead of distinct hours, minutes and seconds as `int`s.

- Another additional constructor that takes a single integer, representing a number of seconds, as its actual argument value provides a convenient way to declare and construct the resultant mean as a time value.

The sub-activities of this lab are to be done in sequence and incrementally. As you accomplish each sub-activity you are to show your revisions to your lab instructor and demonstrate them as well.

Once you are convinced that you have all of the sub-activities completed, or completed as much as you can, you should again review your revisions with your lab instructor and demonstrate the programs. When all activities are successfully completed the given `MeanHHMMSS` program should be able to execute fully and correctly.

As a further check on the behavior of the methods you have developed and revised an additional program, `TimeSpanDriver.java`, is provided for you to run. Only then should you submit the final versions of your revised `TimeSpan.java`, and `TimeSpanOperations.java` source programs to the CMPS 134L CWS for the section you are working in.

```
c:\C134L\Lab11>java TimeSpanDriver
Enter:>12:34:56
12:34:56.equals(12:34:56) yields true
00:00:00.equals(12:34:56) yields false
Enter:>    2   :  34  :   56
02:34:56.equals(02:34:56) yields true
12:34:56.equals(02:34:56) yields false
Enter:>1:2:3
01:02:03.equals(01:02:03) yields true
02:34:56.equals(01:02:03) yields false
Enter:>1234:56:7
1234:56:07.equals(1234:56:07) yields true
01:02:03.equals(1234:56:07) yields false
Enter:>123456:78:90
123457:19:30.equals(123457:19:30) yields true
1234:56:07.equals(123457:19:30) yields false
Enter:>
Done!!!
```

As always, be sure that you have revised and completed the comments at the beginning of each source code file you have revised (providing the names of all team members) before submitting. You are to submit the following source programs:

- `MeanHHMM.java`
- `TimeSpan.java`
- `TimeSpanOperations.java`

Activity 3: Practice-It Self-Checks and Exercises

After logging in to Practice-It, click on the link labeled **Building Java Programs, 4th Edition** and then on the link indicating **BJP Chapter 7**.

Do the following Self-Checks and Exercises and, when finished, show your GTA (graduate teaching assistant) that you've completed all of them correctly. (Go back to the web page that lists all the Chapter 7 Self-Checks and Exercises and refresh it. Any that were answered correctly will be indicated on the page.)

1. **BJP4 Self-Check 7.1 (arrayDeclarationSyntax):** Multiple Choice
2. **BJP4 Self-Check 7.2 (numbersElements):** Provide Syntax
3. **BJP4 Self-Check 7.3 (data Array):** Write code
4. **BJP4 Self-Check 7.6 (fillDataArray):** Provide array element values
5. **BJP4 Self-Check 7.8 (arrayDeclarationSyntax2):** Multiple Choice
6. **BJP4 Self-Check 7.9 (declareArray):** Provide statement
7. **BJP4 Self-Check 7.22 (arrayCodeTracing3):** Provide array element values
8. **BJP4 Self-Check 7.26 (arrayMystery3):** Provide array element values

---

Activity #4: The `CoinJar` Instance Class and the `CoinJarClient` Application Program

Consider the given instance class, `CoinJar.java`, which abstracts a collection of U.S. coins of various denominations. Many people discard the change that accumulates in their pockets or purses into a box or jar, and such a vessel would be a real world manifestation of a "coin pile" as abstracted here. A "Piggy Bank" would likewise be a similar manifestation.

This class is adequately commented and it will compile and run, but since there are several places in it that are stubbed its results are incomplete.

Next consider the given client program, `CoinJarClient.java`, which is complete and which there is no reason for you to change. This program receives the name of a file to be read as a command line argument. (Your lab instructor can show you how to specify the filename in jGRASP.) The file consists of "coin codes", that is '`P`', '`N`', '`D`', '`Q`', '`H`', that stand for Penny, Nickel, Dime, Quarter and Half respectively. The files `Coins00.txt`, `Coins01.txt` and `Coins02.txt` are example files that adhere to this structure and content. Illustrations of the execution of this program with a completed `CoinJar` class, using these two data files are shown.

```
c:\Lab10_20180416>java CoinJarClient Coins00.txt
P (1) yields $0.01, filling 0 rolls.
N (2) yields $0.10, filling 0 rolls.
D (4) yields $0.40, filling 0 rolls.
Q (3) yields $0.75, filling 0 rolls.
H (0) yields $0.00, filling 0 rolls.
  (10) yields $1.26

c:\Lab10_20180416>java CoinJarClient Coins01.txt
P (296) yields $2.96, filling 5 rolls.
N (201) yields $10.05, filling 5 rolls.
D (197) yields $19.70, filling 3 rolls.
Q (280) yields $70.00, filling 7 rolls.
H (26) yields $13.00, filling 1 rolls.
  (1000) yields $115.71

c:\Lab10_20180416>java CoinJarClient Coins02.txt
P (3686) yields $36.86, filling 73 rolls.
N (2527) yields $126.35, filling 63 rolls.
D (2475) yields $247.50, filling 49 rolls.
Q (3333) yields $833.25, filling 83 rolls.
H (324) yields $162.00, filling 16 rolls.
  (12345) yields $1405.96
```

For this activity you are to first carefully read the comments in `CoinJar.java` and study the design of the declarations and implementation given. Correlating the use of these features with that shown in the client program should help you understand how these two parts are used together to perform a useful task.

In particular, note that there are two missing declarations in `CoinJar.java` as noted by the comments:

```
// DECLARATION STUBBED!!!
```

Both of these missing declarations call for arrays to be declared.

The first is for an appropriately named array that holds the capacities of paper "rolls" for each of the corresponding coin denominations. For your information, a "Penny Roll" has a capacity of 50 coins, a "Nickel Roll" has a capacity of 40 coins, a "Dime Roll" has a capacity of 50 coins, a "Quarter Roll" has a capacity of 40 coins, and a "Half Dollar Roll" has a capacity of 20 coins.

The second is for an appropriately named array that holds the counts of the number of coins of each of the corresponding currently in the collection. Note that initially each of these counts should be zero.

Also, in particular, note that four of the methods are likewise stubbed as noted by the comments:

```
//STUBBED!!!
```

Your task is to provide the missing declarations and instructions in the indicated places so that the class performs fully as per its comments and illustrated use.

Once you are convinced that you have the program complete you are to show it to your instructor and demonstrate it before submitting the completed `CoinJar.java` file to the CMPS 134L CWS for the section you are working in.